

REACT

REACT REGELN (KOMPONENTE, HTML, JSX)

- Nur ein Rückgabewert: 1 parent element (Root Element) oder React.Fragment <</>
- Naming der Komponente
 - UpperCamelCase
 - treffende Beschreibung der Komponente
 - Beispiele: LinkButton, InfoTooltip, DraftEditor
- HTML Attribute in JSX mit camelCase (ausser aria-* und data-*)
 - class → className, for in Formularfeldern → htmlFor, stroke-width → strokeWidth
- HTML Elemente geschlossen, z.B. , <input />
- Syntax wie in JS. Wert in (title)

REACT CODE

Demo v19

```
import { useReducer } from 'react';

type ReducerActionTypes = 'increment' | 'decrement';

const reducer = (
  state: { count: number }, // state ist Zahl von counter
  action: { type: ReducerActionTypes }, // + oder -
) => {
  switch (action.type) {
    case 'increment':
      console.log('incrementing');
      return { count: state.count + 1 };
    case 'decrement':
      console.log('decrementing');
      return { count: state.count - 1 };
    default:
      throw new Error('Unknown action type');
  }
};

export const StaticCounter = () => {
  const actions: Record<'type', ReducerActionTypes>[] = [
    { type: 'increment' },
    { type: 'increment' },
    { type: 'increment' },
    { type: 'decrement' },
  ];

  // static use of the reducer
  const finalState = actions.reduce(reducer, { count: 0 });
  return <static reducer result: {finalState.count}</>;
};

export const Counter = () => {
  // use reducer with useReducer hook
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <h3>useReducer</h3>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'decrement' })}>- </button>
      <button onClick={() => dispatch({ type: 'increment' })}>+ </button>
    </div>
  );
};
```

AJAX

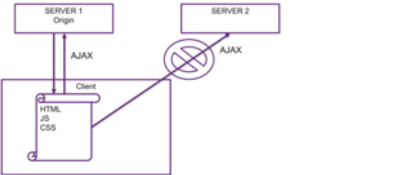
Motivation: User Experience verbessern mit Auto-Complete in Formularen. Nachträgliches Laden von Daten und Updaten von Website.

Nachteile: State der Applikation ist nicht bekannt. Undo/Redo-State (History) wird oft vergessen.

Fetch: Neue Variante von XMLHttpRequest. Fetch Response bietet Methoden an um die Daten vom Body zu interpretieren. Cookies werden nicht automatisch mitgesendet.

```
fetch('/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    email: 'admin@admin.ch',
    pwd: '12345678'
  })
}).then(function (res) {
  console.log(res);
});
```

SOP/CORS: Same-Origin-Policy erlaubt XMLHttpRequest nur zur Origin. Cross-Origin Resource Sharing ist ein Mechanismus, um Cross-Site-Requests zu ermöglichen. Der Ziel-Server erlaubt dem Client den Zugriff erlauben.



EXPRESS

CONTROLLER

Beispiel:

```
import express from 'express';
const app = express();
app.listen(3000, function () {
  console.log('Example app on port 3000!');
});
```

Middleware: Express nutzt Middleware für die Request Bearbeitung. Middleware ist ein Stack von Anwendungen welche für ein Request ausgeführt wird.

Request → Middlewares → Response

Middleware registrieren: Mit app.use(...) wird eine neue Middleware registriert. Die Reihenfolge der Registrierung bestimmt die Ausführungsreihenfolge.

```
import express from 'express';
import bp from 'body-parser';
const app = express();
const router = express.Router();
app.use(express.static(__dirname + '/public'));
```

```
app.use(bp.urlencoded({extended: false}));
app.use(router);

Middlewares-Sammlung Connect: Connect definiert die Middleware Logik und eine Sammlung von Middlewares (z.B body-parser, cookie-parser)

Router-Middleware: Middleware befindet sich auf dem Express Objekt.

  - router.all(path, [callback, ...] callback)
    - wird unabhängig von der HTTP Methode aufgerufen. Dynamische Werte möglich (/order/{id})
  - router.METHOD(path, [callback, ...] callback)
    - wird aufgerufen, falls die jeweilige HTTP Methode verwendet wurde

router.get('/', function (req, res) {
  res.send('hello world');
});

  - router.route(path)
    - kann benutzt werden um für einen Path verschiedene Methoden zu gruppieren

app.route('/book').get(function (req, res) {
  res.send('Get a random book');
}).post(function (req, res) {
  res.send('Add a book');
});
```

Static-Middleware: Statische Files ausliefern. Mehrere static-routes möglich

Custom-Middleware: Hat 3 Parameter request, response und next. Next kann aufgerufen werden, um die nächste Middleware aufzurufen.

```
function myLogger(options) {
  options = options ? options : {};
  return function myInLogger(req, res, next) {
    console.log(req.method + " " + req.url);
    next();
  }
}
```

Error-Middleware: Bearbeitet Errors, welche von den Middlewares generiert wurden. Muss 4 Parameter haben und als letztes registriert werden. Wird aufgerufen, falls ein Error-Objekt dem Next-Callback übergeben wird.

```
app.use(function (err, req, res, next) {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
```

MODEL

Ziel: Die Daten sollten in einem Modul verwaltet und abgespeichert werden. (z.B mit Array, JSON, NoSQL oder SQL-DB).

NoSQL DB mit needs: NoSQL Datenbanken sind Dokumenten-basierend. Jedes Dokument beinhaltet alle Daten, welche notwendig sind. Relationen können über 'keys' manuell erstellt werden.

Beispiel:

```
import Datastore from '@seald-io/medb';
const db = new Datastore({
  filename: './data/order.db',
  autoload: true });
db.insert(order, function (err, newDoc) {
  console.log('insert');
  if (callback) {
    callback(err, newDoc);
  });
};
```

db.find(id, function (err, docs) {
 callback(err, docs);
});

db.update({ id: id,
 { \$set: { 'state': 'DELETED' } },
 (return)UpdatedDocs: true,
 function (err, numDocs, doc) {
 callback(err, doc);
 });

VIEW

Mit Template Engines kann HTML-Code generiert werden. «Einfache» Sprache um (komplexere) Templates zu unterstützen. Single Page Frameworks und Web-Frameworks nutzen Template Engines um Templates zu rendern. Trennt Controller und View.

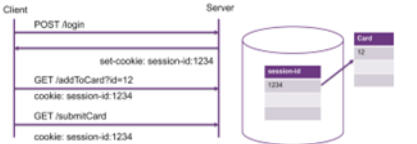
Beispiel:

```
res.render('template', {
  description: 'List of some Movies',
  items: songs});

<figure>
  <ul>
    {({each item})}
  </ul>
  <h3>{title}</h3>
  <p>{[artist]}</p>
  </li>
  {({each})}
</ul>
<figcaption>
  <p>{[description]}</p>
</figcaption>
</figure>
```

SESSION

Cookie: Ein Cookie repräsentiert ein kleines Stück Information. Der Server schreibt ein Cookie auf einen Client (set-cookie: ...). Client schickt alle Cookies für aktuelle Seite zurück und Server (cookie: ...).



Session: HTTP-Stateless umgehen. Daten werden Server-seitig einem Benutzer zugeordnet.

Authentifizierung: Wer bin ich? (Passwort, Handy, Biometrie)

Autorisierung: Was darf ich?

Tokens: Ziel ist ein Stateless Server. Bei jeder REST Anfrage muss ein Token mitgegeben werden (Signatur, Ausstell-/Abfauddatum). Vorteil ist dass jede Anfrage zu einem beliebigen Server gesendet werden kann. Nachteil ist dass Token geklaut werden kann.

TOKEN GENERIERUNG: Bei einer Route müssen die persönlichen Daten angegeben werden. Falls korrekt, wird ein Token generiert und an den Client geschickt. Verschiedene Libraries wie OAuth, Express-JWT, etc.

TYPESCRIPT (TS)

Motivation: TS hilft mittels «statischer Analyse», entdeckt Typen, erlaubt Spezifikation von Typen für Variablen/Parameter etc.

Typ-Inferenz: boolean, number, string, null, undefined und any. Any kann beliebigen Wert annehmen und kann einen beliebigen anderen Variablen zugewiesen werden.

Strict Mode: Keine untypisierten Variablen, null und undefined ist nicht mehr Teil der Basistypen und können auch nicht mehr zugewiesen werden.

Variablen Deklarationen: Globale Variablen aus nicht TS-Files können mit dem Keyword declare deklariert werden. Variablen können als unknown deklariert werden, dessen Typ später definiert wird.

```
declare let myMagicVar: string;

Auf Tupeln wird keine Typen-Inferenz angewendet, bei Enums wird es. Bessere Alternative zu Enums sind String Literal Union Type.

let myInferredNumArray = [1, 2, 3];
let myNotInferredTupel = [1, 'abcd'];

enum StrColor { Red = 'red', Green = 'green' };
let c: Color = StrColor.Green;
type StrLitColor = 'red' | 'green';
let c3: StrLitColor = 'green';
```

Union Type:

```
type StringOrNumberType = string | number;
let myVar2: StringOrNumberType;

Funktionen: Function Overloading erlaubt. Parameter können Default Value und optional sein

function f(sn: number | string | string[], ns?: number): string {
  return String(sn) + String(ns || '');
}

// Funktion als Parameter
function f2(numArray: number[],
  numFun: (prev: number,
  curr: number) => number): number {
  return numArray.reduce(numFun);
}
```

Klassen: Properties der Instanzen und der Klasse (static) werden im Kontext der Klasse definiert. Methoden und Properties können mit den Zusätzen 'private' und 'readonly' versehen werden.

```
class SportsCar {
  constructor(
    public make: string,
    public color: SportsCarColor
  ) {
    // kein code nötig
  }
}
```

Interfaces: Interfaces (Typen) können in Deklaration von Klassen genutzt werden und darf mehr als ein Interface implementieren. Readonly Variablen können nicht verändert werden und *? bedeutet optional.

```
interface IPoint {
  readonly x?: number;
}

class Item {
  constructor(public description: string) {}
}

class POI extends Item implements Point {
  constructor(public x: number,
    public y: number,
    description: string,
    public likes?: number) {
    super(description);
  }
}
```

Typen vs Interface: Vielfach lassen sich strukturierte Typen sowohl als Interface als auch als Type definieren. Aber nur ein interface kann unter gleichen Namen erweitert werden und nur ein *Type kann Werte spezifizieren.

Type Assertion: Type Assertion erlauben Spezialisierung und Generalisierung des Typs, aber kein Casting auf einen inkompatiblen Typ. Mit keyof lässt sich ein Typ aus einem anderen Typen ableiten

```
const myCanvas = document.getElementById('1na') as HTMLCanvasElement;
const x = \"hello\" as number; //mpfh\\V\\V nicht erlaubt

type Point = { x: number; y: number };
type PK = keyof Point;

#strong[Template Literal Types] Mit Template Literal Types lassen sich
<literal> Types aus anderen Typen ableiten.

type Shipper = `UPS` | `FEDEX` | `DHL` | `Overnight` | `Priority` | `Economy`;
type PostMethod = `\\${Shipper}\\-${TrackingType}`;
```

Generics: Mit Generics lassen sich Funktionen und Strukturen beschreiben die Wiederverwendung mittels 'Composition' ermöglichen.

```
interface PointList<Item>Type {
  ItemList: ItemType[];
}
```

```
function filteredPointList<Item>Type(
  pList: PointList<Item>Type,
  incPred: (Item: ItemType) => boolean): ItemType[] {
  return pList.ItemList.filter(incPred);
}

Index.js
import { app } from './app.js'
// const app = (await import('./app')).app; // lazy loading
const hostname = '127.0.0.1';
const port = 3001;
app.listen(port, hostname, (error) => {
  if (error) {
    console.error(error);
  } else {
    console.log(`Server running at http://${hostname}:${port}`);
  }
});
```

```
app.js
import express from 'express';
import path from 'path';
import session from 'express-session';
import expbs from 'express-handlebars';
// (more imports inline for readability)
export const app = express();
const hbs = expbs.create({
  extname: '.hbs',
  defaultLayout: 'default',
  helpers: {
    ...helpers
  }
});

// import { helpers } from './utils/handlebar-util.js'
export const helpers = {
  'if_eq': function (a, b, opts) {
    if (a === b)
      return opts.fn(this);
    else
      return opts.inverse(this);
  }
}
```

```
app.engine('hbs', hbs.engine);
app.set('view engine', 'hbs');
app.set('views', path.resolve('views'));

views/index.hbs
<{{data}}
views/layouts/default.hbs
<doctype html>
<html lang=en>
<head>
  <meta charset=UTF-8>
  <meta name=viewport>
    content=width=device-width, user-scalable=no, initial-scale=1.0,
    maximum-scale=1.0, minimum-scale=1.0>
  <meta http-equiv=X-UA-Compatible content=ie=edge>
  <titleToday/!title>
  {{#if_eq theme 'dark'}}
    <style>
      body {
        background: black;
        color: white;
      }
    </style>
  {{/if_eq}}
</head>
<body>
  {{{body}}}
</body>
</html>
```

```
app.use(express.static(path.resolve('public')));
app.use(session({secret: 'casduichasidbmuewzfrinsdcvjkadhsfuzifzifhida',
  resave: false, saveUninitialized: true}));
app.use(sessionUserSettings);
// import { sessionUserSettings } from './utils/session-middleware.index.js'

export const sessionUserSettings = (req, res, next) => {
  const userSettings = req.session.userSettings || {orderBy: 'title',
  orderDirection: -1, theme: 'dark'};
  const {orderBy, orderDirection, theme} = req.query;

  if (theme) {
    userSettings.theme = theme;
  }

  if (orderBy) {
    userSettings.orderby = orderBy;
  }

  if (orderDirection) {
    userSettings.orderDirection = orderDirection;
  }

  req.userSettings = req.session.userSettings = userSettings;
  res.locals = req.userSettings; // visible within views
  next();
};
```

```
app.use(express.urlencoded({extended: false}));
app.use(express.json());

app.use('/', indexRoutes);

// import { indexRoutes } from './routes/index-routes.js';
import express from 'express';
const router = express.Router();
import {indexController} from './controller/index-controller.js';

export class IndexController {
  index(req, res) {
    res.render("index", {data: 'Hello World'});
  };
  export const indexController = new IndexController();

  router.get('/', indexController.index.bind(indexController));
  export const indexRoutes = router;
```

EXPRESS TESTAT CODE

```
index.js
see express.js demo
app.js
import express, { NextFunction, Request, Response } from 'express';
import path from 'path';
import cors from 'cors';

import { userRoutes } from './routes/user-routes';
import { accountRoutes } from './routes/account-routes';
import { transactionRoutes } from './routes/transaction-routes';
import { CONFIG } from './config';
import { expressJwt } from 'express-jwt';
import { HttpError } from './services/http-error';

export const app = express();

app.use(cors());
app.use(express.static(path.resolve('public')));
app.use(express.json());

app.use(
  expressJwt(CONFIG.jwt_validate).unless({
    path: [/^\\/login$/, /^\\/register$/,
  ]},
);

// app.use('/users', userRoutes);
// app.use('/accounts', accountRoutes);
app.use('/transactions', transactionRoutes);

app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
  if (err instanceof HttpError) {
    return res.status(err.status).json({ message: err.message });
  }
  if (err.name === 'UnauthorizedError') {
    return res.status(401).json({ message: 'No token / Invalid token provided' });
  }
  return res.status(400).json({ message: 'something went wrong', err });
});

routes/transaction-routes.ts
import express from 'express';
import { transactionController } from '../controller/transaction-controller';

const router = express.Router();

router.post('/', transactionController.create);
// router.get('/', transactionController.getTransactions);
// router.get('/:accountNr', transactionController.getTransactions);

export const transactionRoutes = router;
controller/transaction-controllers
import { Request, Response } from 'express';
import { HttpError } from '../services/http-error';
import { SchematUtil } from '../utils/scheme-util';
import { AddTransactionSchema, FindTransactionSchema, transactionService } from './services/transaction-service';

export class TransactionController {
  public create = async (req: Request, res: Response) => {
    const auth = req.headers.authorization;
    if (!auth) { throw new HttpError(401, 'no token provided'); }
    if (!req.auth.userId) { throw new HttpError(401, 'Unauthorized'); }

    const data = SchematUtil.parseOrThrow(AddTransactionSchema, req.body);

    const userID = req.auth.userId;

    // import { accountService } from './../services/account-service';
    const isOwner = await accountService.isOwner(data.from, userID);

    if (!isOwner) { throw new HttpError(402, 'Incorrect user'); }
    await transactionService.create(data);
    return res.status(200).json({ success: true });
  };

  services/transaction-service.ts
  import Datastore from '@seald-io/medb';
  import { z } from 'zod';
  import { accountService } from './account-service';
  import { HttpError } from './http-error';

  const TransactionSchema = z.object({
    from: z.number(), // can be float
    amount: z.number().int(),
    date: z.date(), to: z.date().optional().default(new Date());
  }); type Transaction = z.infer<typeof TransactionSchema>;

  export const AddTransactionSchema = z.object({
    amount: z.number().int().positive(), from: z.string().optional().default('');
  }); type AddTransactionSchema = z.infer<typeof AddTransactionSchema>;

  type FindTransactionResult = {
    docs: Transaction[]; count: number; skip: number; docCount: number;
  };

  export const FindTransactionSchema = z.object({
    accountNr: z.coerce.number(),
    count: z.coerce.number().int().positive().optional().default(10),
    skip: z.coerce.number().int().nonnegative().optional().default(0),
  }); type FindTransaction = z.infer<typeof FindTransactionSchema>;

  export class TransactionService {
    private db: Datastore<Transaction>;

    constructor() { this.db = new Datastore({}); }

    async create(data: AddTransaction): Promise<void> {
      const from = await accountService.getData.from;
      if (from.balance < data.amount) {
        throw new HttpError(400, 'Insufficient funds');
      }
      // throw error if amount < 0, sending to yourself..
      const newFromBalance = from.balance - data.amount;
      const to = await accountService.getData.to;
      const newToBalance = to.balance + data.amount;

      const data = new Date();
      await this.db.insertAsync({ from: data.from, ... });
      await accountService.update(from.accountNr, { balance:
      newFromBalance });
      await accountService.update(to.accountNr, { balance:
      newToBalance });
    } // transactionService.create

    async find(data: FindTransaction): Promise<FindTransactionResult> {
      const filter = {
        $or: [{ from: data.accountNr }, { to: data.accountNr }],
      };

      const limit = data.count < 5 ? data.count : 5;

      let transactions: Transaction[] = await this.db
        .findAsync(filter)
        .sort({ date: -1 })
        .skip(data.skip)
        .limit(limit);

      const transactionCount = await this.db.countAsync(filter);

      return {
        docs: transactions,
        count: transactions.length,
      };
    }
  }
}
```

```
skip: data.skip,
doCount: transactionCount,
});
} // transactionService.find
} export const transactionService = new TransactionService();
```

ACCESSIBILITY

Farbkontrast: Wichtig für Personen über 50

Accessibility:

- Bilder sollen immer einen Alt-Text haben
- Zoom sollte nicht unterbunden werden
- Animationen sollten abstellbar sein (Verringerung von Ablenkungen und Verhinderung von Epilepsie und Migräne)
- Alle wichtigen Input Elemente sind fokussierbar in der richtigen Reihenfolge mit Tastatur
- Screen Reader soll keine Heading Levels auslassen, Semantic Elements richtig nutzen, Skip-Links am Anfang der Seite und lang Attribut korrekt setzen
- Entwickler sollten die Verwendung des korrekten semantischen HTML-Elements der Verwendung von ARIA vorziehen
- Tabellen sollten mit Headings for Rows und Columns ausgestattet sein und Captions haben

SECURITY

OWASP Top 10:

Cross-Site-Scripting	Replay Attack
Remote Code Execution	Cryptographic Failures
Insecure Direct Object References	Identification & Authentication Failure
Cross-Site Request Forgery	

Cross Site Scripting (XSS): Website besitzt eine XSS-Verwundbarkeit, wenn es möglich ist den Server so zu manipulieren, dass Schadcode (JavaScript) eines Angreifers an Nutzer ausgeliefert wird und im Browser dieser Nutzer ausgeführt wird. Gegenmassnahme mit Input Sanitation.

Remote Code Execution: Webserver besitzt eine Code Injection "Vulnerability" wenn ein Angreifer den Server dazu bringen kann vom Angreifer eingeschleusten Code zum Ausführen zu bringen.

Broken Access Control: Beim behandeln von Formular-Submission-Requests (GET und POST) sollte überprüft werden, dass dies von einem vom Server für den Nutzer «frisch» ausgelieferte Formulare stammt.

Cryptographic Failures: Passwort oder Token wird nicht verschlüsselt übertragen. Unabhängig davon ob Query-Body/Request-Parameter.

Identification & Authentication Failure: Bei Problemen bei der Authentisierung und dem Session Management können externe Angreifer oder Angreifer mit einem validen Login auf Informationen zugreifen, welche nicht für sie bestimmt sind

Sign-up Form Best Practices:

- Use meaningful HTML elements: form, input, label and button
- Label each input with a label
- Use element attributes to access built-in browser features: type, name, autocomplete, required.
- Use autocomplete="new-password" and id="new-password" for the password input in a sign-up form, and for the new password in a reset-password form.
- Provide Show password functionality.
- Don't double-up inputs. Don't force users to enter emails or passwords twice.

Content Security Policy (CSP): CSPs ermöglichen die Ausführung von bösartigem Code auf Webseiten zu verhindern. Quellen von Ressourcen (Skripten, Bilder etc.) können eingeschränkt werden. CSPs werden im HTTP Header definiert

Cross-Origin Resource Sharing (CORS): CORS Header ermöglichen Ressourcen (wie z.B. Bilder, Skripte oder Daten) von einem anderen Ursprungsort als der eigenen "Origin" zu laden.

TESTING

Unit Tests: Getestet werden einzelne "Units" wie Klassen, Module (meist ein File). Automation ist relativ einfach. Herausforderung ist die Isolation der Units, asynchrone Operationen, Testdaten-generierung (z.B. Fuzzing)

Integrations-tests: Getestet wird das Zusammenspiel 2 oder mehr "Units". Automation meist möglich. Herausforderungen ist die Isolation der Units, asynchrone Operationen, Simulation Browser & Benutzerinteraktion, Test mit Datenbank, Testdatengenerierung (z.B. Fuzzing)

Funktionstests: Getestet wird ob sich das System entsprechend spezifizierte funktionale Anforderungen (Use-Cases, User Stories, ...) verhält. Automation möglich mit speziellen Tools

(Visuelle) Regressionstests: Getestet wird ob Veränderungen im Code zu (unerwarteten) Änderungen im Verhalten (oder UI) führen. Automation möglich mit speziellen Tools

Funktionale Systemtests: Getestet wird das Zusammenspiel aller Systemkomponenten in der Zielumgebung Automation meist nur in Teilen möglich. Herausforderungen: Realistische aber vorhersagbare Umgebung

TOOLS

Test-Runner: Ein Rahmen der Tests entgegennimmt, ausführt und die Resultate anzeigt. Beispiele: Ava CLI, Jasmine, Jest, Mocha, Cypress

Assertion Library: Code zur Ausführung einzelner Tests (Unterstützung Testing Patterns) Beispiele: Assert, Ava Power-Assert, Chai, Expect.js

Mocking Library: Separierung von Units / Erstellung von Mocks etc. Beispiele: Expect.js, Proxyquire, Sinon.js

DOM Handling : Cypress, JSDom, Puppeteer (Headless), Playwright, Storybook (Regression), Enzyme (React)

Mocha API: Aufruf einer umfassenden Funktion describe() mit zwei Argumenten.

```
describe('Array', function () { // Test Suite
describe('#indexOf()', function () {
beforeEach(function () {
this.testArray = [1, 2, 3]; // Test Setup
});
```

```
// Test Case
it('return -1 when value not here', function () {
```

```
// Assertion
expect(this.testArray.indexOf(4)).to.equal(-1);
```

```
});
```

```
});
```

Unit Testing: Alles testen das kaputt gehen könnte. Alles testen das kaputt gegangen war. Neuer Code ist sus bis unschuldig!Test bewiesen.

SPICK ANDRIN KESSLER

Unit Test: einzelne Klassen/Module (meist ein File) werden getestet, einfache Automation, Herausforderungen: Units isolieren, asynchrone Operationen, Testdaten-Generierung (z.B. Fuzzing) Integration Test: Zusammenspiel von 2 oder mehr Units wird getestet, Automation meist möglich, Herausforderun-gen: gleiche wie bei Unit Tests, Simulation Browser & Benutzerinteraktion, Test mit Datenbank End-to-End Test: Integration Test über alle Layer Anderes: Smoke, Static (TypeScript), Regression, Load, Performance, Endurance, Chaos, Security, Usability Test-Runner: Rahmen, der

Tests entgegennimmt, ausführt und Resultate anzeigt, z.B. Mocha, Cypress Assertion Library: gut lesbare Annahmen schreiben, die nicht verletzt werden sollten, z.B. Assert, Chai Mocking Library: Separierung von Units, Erstellung von Test-Doubles insb. Mocks, z.B. Proxyquire, Sinon.js Test Double Pattern: Bei Unit Tests hat man zum Teil ein Dependent On Component (DOC) an der Unit, mit Sinon kann man einen vorbereitenden Input (Stub/Fake) fürs SUF um überprüfbareren Output (Spy/Mock) liefern Eigenschaften guter Unit Tests: Output statt Internes testen, jeder Test hat voraussetzbare & deklarative Struktur, nur eine Aktion und der Output davon testen, keine Daten mit anderen Tests teilen, alles Nötige zum Verstehen der Absicht muss im Test sein Unit Test Smells: Hard-to-Test Code, viele Bugs bei formalem Testing & Produktion, Tests fallen wenn SUTs so verändert werden, dass die Tests nicht beeinflusst werden sollten, Mail-so-mail-so Tests, Conditionals in Tests, Test-Code Duplikate, langsame Tests, schwierig zu verstehende Tests, Test-Logik im Bro-Code, schwer zu sagen welches Assert im Test failed, keine Tests

```
const { expect } = require('chai');
describe('Array', function () {
describe('#indexOf()', function () {
beforeEach(function() { this.testArray = [1, 2, 3]; });
it('should return -1 when the value is not present',
function() {
expect(this.testArray.indexOf(4)).to.equal(-1); }); }); });
```

EXPRESS

meistgenutztes, etwas veraltetes Web Framework Middlewares: werden mit use() registriert, Registrierungs-reihenfolge = Ausführungsreihenfolge, next(), zeigt jeweils auf die nächste MW, offizielle Middlewares: body-parser, compression, connect, timeout, cookie-parser, cookie-session, csrf, errorHandler, express-session, method-override, morgan, response-time, serve-favicon, serve-index, serve-static, vhost Error-Middleware: bearbeitet Errors, welche von den MWs generiert wurden, sollte zuletzt registriert werden, mehrere sind möglich, wenn next() mit einem Error-Objekt aufgeru-fen wird, werden keine MWs mehr ausgeführt und stattdessen die Error-MWws ausgeführt, 4 Params: Body-parser: parse Request-Bodies und stellt die Variablen unter req.body zur Verfügung Cookie-parser: Parsst den Cookie Header und stellt die Cookies unter req.cookies zur Verfügung Express-session: req.session ist ein Objekt, das temporäre Infos über aktuelle User Sessions speichert Error-handler: development-only, schickt error stack traces zurück zum Client je nach Konfiguration Csrf: verhindert Cross-Site Request Forgery (CSRF) Angriffe durch CSRF Tokens in Cookies express.static: Bereitstellung statischer Files wie Bilder, CSS-/JS-Files, Beispiel am Anfang des Kapitels express.router: Router fürs Request-Handling, Beispiel router const und use(router) am Anfang des Kapitels Request Parameter: Ressourcenidentifikation, Abbildung hierarchischer Datenstrukturen URL Query Parameter: Such-/Filter-Operationen, Steue-rung Pagination, Beispiel für «/search?ti=-b» NEDB: dokumentorientierte, file-backed Datenbank Cookies: im Browser, können persistent sein, automatisch bei jeder Anfrage gesendet, für Sitzungsverwaltung, Personalisierung, Tracking und Analyse Session Objekte: auf dem Server, Lebensdauer bis Ablauf oder Logout, nur innerhalb von Express verfügbar, für Sitzungsverwaltung und speichern temporärer Infos über aktuelle User Sessions REST: Verschiedene Paths für verschiedene Ressourcen, HTTP-Verben & JSON korrekt verwenden, stateless: Session durch JWT Token im Header ersetzen Neben json(): array(Buffer), blob(), formData() & text() Web-Sockets: bidirektionale, dauerhafte Kommunikations-Verbindung zwischen Client & Server, geringe Latenz, wenig Overhead, komplexe API (locket), für Echtzeit-Apps: Chat, Game, Live-Ticker, Docs-Collab, Push-Nachrichten, interaktive Video- / Audio-Streams Handlebars.js: Template Engine, um auf der Server-Side HTML Code zu generieren

TS

TypeScript-ESLint: statische Analyse des Codes zur Fehlerfindung, Installation: "npm install --save-dev eslint @eslint/js @types/eslint" _js typescript typescript-eslint", Lint ausführen: «npm eslint.» , eslint.config.js

Prettier: formatiert Code beim Speichern, setzt konsistente Stilregeln durch, ESLint berücksichtigt auch Prettier Regeln, VS Code Extension, .prettierrc: Basis-Typen: boolean, number, string, null, undefined (no value), any (don't care), unknown (don't know), void, never, array, tuple, enum, union

```
declare let v3: string; let v1 = «Hayes»; let v2: number; v2 = 1; interface User { name: string; readonly id: number; } class UserAccount implements User { constructor() public name: string; public id: number } } const user: User = { name: v1, id: v2; } const user = { name: v1, id: v2; } function deleteUser(user: User) { ... } function getAdminUser(): User { ... } type WindowStates = «open» | «closed» | «minimized»; type WindowStatesLong = $(WindowStates)-window; let myInferred-NumArray = [1, 2, 3]; let myNumArray: Array = [1, 2, 3]; let myTuple: (number, string) = [1, «abcd»]; let unknownVar: unknown; let numberVar = 1; unknownVar = 3; if (typeof unknownVar == «number») numberVar = unknownVar function add(s1: string, s2: string): string { function add(n1: number, n2: number): number { function combine(aFunction(s: number | string) { string = «» , n2: number): string { return String(n1)+String(n2) | «»}; function numberAppliator(numArray: number[], numFun: (prevRes: number, current: number) => number): number { return numArray.reduce(numFun); } interface PointList { itemList:T[]; } } const myCanvas = document.getElementById('main_canvas') as HTMLCanvasElement;
```

ACCESSIBILITY

Rechtlich: UNO-Behindertenrechtskonvention (UNO-BRK), gleichberechtigte Möglichkeiten im Zugang & Benutzung von Informations- & Kommunikationssystemen Häufigste Fehler: Text zu wenig Kontrast, kein alt Text, leere Links/Buttons/Doc-Sprache, fehlende Form Labels Farbenblindheit: prüfen mit Dev-Tools (Rendering), Doppel-Codierung notwendig (Farbe & Form/icon/Text), kritische Informationen nicht nur mit Text codieren, farbenblind-freundliche Farbpaletten wählen Farbkontrast: prüfen mit Dev-Tools (Color Picker) & Lighthouse / Firefox Report, Zielgruppe erweitern mit WCAG Konstan: AA (5:7:1-50+), AAA (15:9:1-80+) Zoombarkeit: Zoom nicht unterbinden, Falsch: user-scalable=0/no, maximum-scale=1 Animationen: sollten abstellbar sein, Verringerung der Ablenkung bei Konzentrationsstörung, Verhinderung der Auslösung von Epilepsie und Migräne Tastatur-Bedienbarkeit: alle wichtigen Input-Elemente sind in der richtigen Reihenfolge fokussierbar, verboten: float, flex-direction: -reverse, Standard Input Controls nutzen: a, button, input[type=«...»], textarea, zusätzliche Elemente können mit «tabindex=0» fokussierbar gemacht werden, Elemente jeder Seite können mit «autofocus» versehen werden Screenreader-Optimierung: keine Headings Level auslassen, semantische Elemente richtig nutzen, Skip-Links am Anfang der Site, lang Attribut korrekt setzen ARIA Attribute: lösen Accessibility Issues, die nicht mit nativem HTML gelöst werden können, heissen jeweils aria-, wichtigste: hidden, label, expanded, current, labelledby, controls, haspopup, invalid, live, required Tabellen: Heading für Rows & Columns mit <th> ausstatten, alternativ <td role="columnheader"/rowheader"> Allgemeines: alt Tag bei Bildern, leer wenn dekorativ, logische Reihenfolge auch ohne CSS, semantische Struktur, Multimedia: 2-Sinne Prinzip, Anzeige-Art kann angepasst werden (Textgrösse, Animationen, Zeitlimits), Überschriften/Form Labels/Linktexte sind verständlich, auf Formatwechsel (z.B. PDF) wird hingewiesen, konsistente Navigation, valides HTML, kompatibel mit versch. I/O Geräten, Hilfestellung bei Interaktionen, PDFs auf PDF

SECURITY

Stored XSS: Server kann so manipuliert werden, dass JS Code an die Browser der Opfer ausgeliefert & ausgeführt wird, Massnahmen: User-Input muss beim Output encoded werden, z.B. mit 3x {} in Handlebars, oder User-Input was sanitized, z.B. mit xs oder dompurify Library JS Injection: dem Server kann JS Code gesetzt werden, welcher ihn im Code mit eval()/setInterval()/setInterval()/ Function() ausführt, Massnahmen: stattdessen parseXXX()/JSON.parse() verwenden, globale Scopes & Variablen reduzieren, rechen-intensive Tasks mit childprocess.spawn auslagern, Node kein Root-Prozess Insecure direct object references: durch Manipulation der URL lassen sich sensible Daten ohne Authentifizierung oder Autorisierung aufrufen, Massnahmen: bei allen Sites mit sensiblen Daten sicherstellen, dass der User eingeloggt und berechtigt ist

CSRF

Cross Site Request Forgery: Angreifer bringt Nutzer mit gültiger Session dazu, auf einer Site ein gefälschtes Formular an die richtige Site zu submitten, Massnahmen: Formular-Submission-Requests (GET/POST) müssen geprüft werden, ob sie von einem vom Server frisch ausgelieferten Formular stammen (mit CSRF Token) Replay attacks: z.B. bei Quiz wird mit Formular Aufgabe und

User-Input-Lösung submitted, wenn richtig gibt es einen Punkt, das kann replayed werden, sollte aber nicht gehen, da der User dann unrechtmässig Punkte erhält, Massnahmen: CSRF Token 1x gültig, bereits gelöste Aufgaben speichern, letzte Aufgabe in Session speichern Cryptographic failures: Crypto schlecht eingesetzt, Massnahmen: https nutzen, keine geheimen Infos in Query-Params (Leaks in Server Logs oder Browser History), Authentication Service nutzen Identification & Authentication failures: z.B. isSession Timeout zu lang, User bleibt auf öffentlichem PC einge-logged, Massnahme: Session Timeout sinnvoll setzen CORS Header: ermöglicht Laden von Ressourcen (Bilder, Scripts, usw.) von anderem Ursprungsort als eigener Origin, man beschränkt, welche Websites auf seine Res-sourcen zugreifen dürfen

CSP Header: Content Security Policy, Beispiel schränkt Kommunikation auf Source Domain/api.ch ein, externe & Inline-Scripts sind verboten, aktivieren mit unsafe-inline Weiteres: DNS Prefetching & Client Caching deaktivieren, Public Key Pinning Headers, Strict-Transport-Security-Header (HSTS) setzen, X-XSS-Protection Header, Safe-Rexes verwenden (DDoS), Parameter pollution Cookies: Secure Attribut: Cookie wird nur über HTTPS versendet, nicht HTTP (ausser localhost), HttpOnly Attribut: deaktiviert Zugriff auf Cookies aus JS